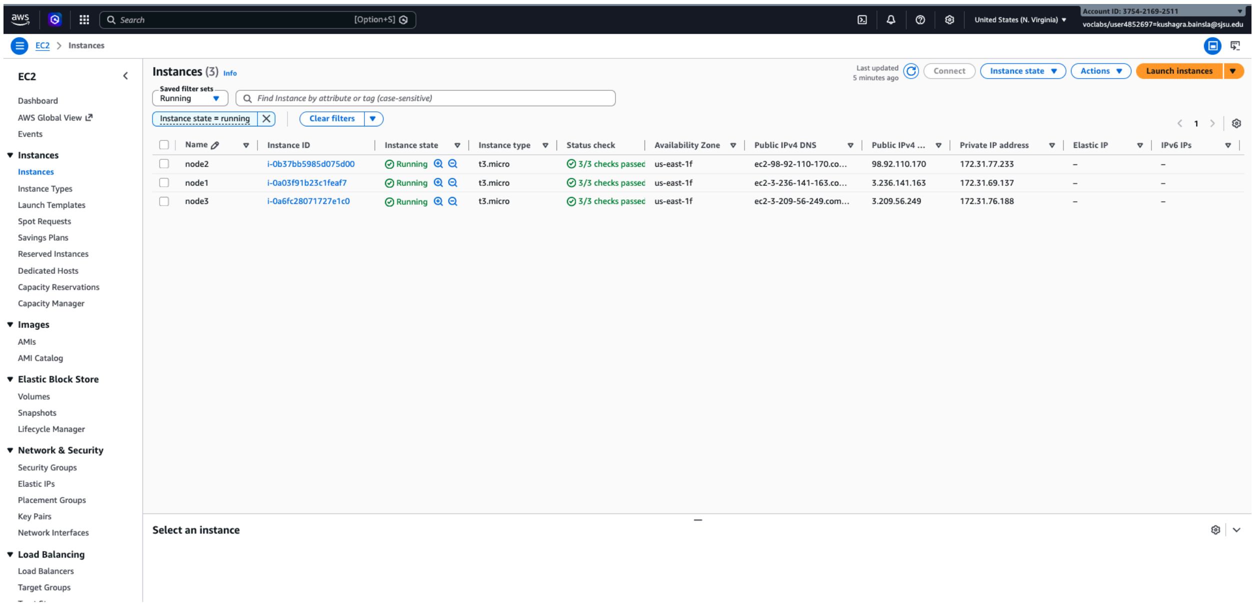
Task 1


Search
[Option+S]
United States (N. Virginia)
Account ID: 3754-2169-2511
voclabs/user4852697=kushagra.bainsla@sjsu.edu
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Instances (3) Info
Last updated 5 minutes ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Running
Find Instance by attribute or tag (case-sensitive)
Instance state = running
Clear filters
Name | Instance ID | Instance state | Instance type | Status check | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Private IP address | Elastic IP | IPv6 IPs
node2 | i-0b37bb5985d075d00 | Running | t3.micro | 3/3 checks passed | us-east-1f | ec2-98-92-110-170.co... | 98.92.110.170 | 172.31.77.233 | – | –
node1 | i-0a03f91b23c1feaf7 | Running | t3.micro | 3/3 checks passed | us-east-1f | ec2-3-236-141-163.co... | 3.236.141.163 | 172.31.69.137 | – | –
node3 | i-0a6fc28071727e1c0 | Running | t3.micro | 3/3 checks passed | us-east-1f | ec2-3-209-56-249.com... | 3.209.56.249 | 172.31.76.188 | – | –
Select an instance

Task 2

```
● ➜  take_home_midterm git:(main) ✗ clear
● ➜  take_home_midterm git:(main) ✗ chmod 400 nosql-lab.pem
○ ➜  take_home_midterm git:(main) ✗ ssh -i "nosql-lab.pem" ec2-user@98.92.110.170

The authenticity of host '98.92.110.170 (98.92.110.170)' can't be established.
ED25519 key fingerprint is SHA256:DM5qUSCt+qSd3NZ3Gtr/sx+2TajVB45Av2x/NPXhaWY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '98.92.110.170' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-77-233 ~]$
```

```
○ ➜  take_home_midterm git:(main) ✗ ssh -i "nosql-lab.pem" ec2-user@3.236.141.163

The authenticity of host '3.236.141.163 (3.236.141.163)' can't be established.
ED25519 key fingerprint is SHA256:CeSt6XJre2HsSTKlu4FTLM3Gqc8EFiqBf3ZKyn372/E.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.236.141.163' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
```

```
○ ➜  take_home_midterm git:(main) ✗ ssh -i "nosql-lab.pem" ec2-user@3.209.56.249
The authenticity of host '3.209.56.249 (3.209.56.249)' can't be established.
ED25519 key fingerprint is SHA256:bRDPKIjXOejDpdUzRQIlkdXSsjUEjQ6P2CEThgXdLmE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.209.56.249' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
```

Task 3

```
Installed:
  mongodb-database-tools-100.16.0-1.x86_64                          mongodb-mongosh-2.8.2-1.el8.x86_64
  mongodb-org-7.0.31-1.amzn2023.x86_64                              mongodb-org-database-7.0.31-1.amzn2023.x86_64
  mongodb-org-database-tools-extra-7.0.31-1.amzn2023.x86_64         mongodb-org-mongos-7.0.31-1.amzn2023.x86_64
  mongodb-org-server-7.0.31-1.amzn2023.x86_64                       mongodb-org-tools-7.0.31-1.amzn2023.x86_64

Complete!
db version v7.0.31
Build Info: {
    "version": "7.0.31",
    "gitVersion": "6a3bdfa2794c261d3ce011c74f818e970e563609",
    "openSSLVersion": "OpenSSL 3.5.5 27 Jan 2026",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "amazon2023",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
```

```
Installed:
  mongodb-database-tools-100.16.0-1.x86_64                          mongodb-mongosh-2.8.2-1.el8.x86_64
  mongodb-org-7.0.31-1.amzn2023.x86_64                              mongodb-org-database-7.0.31-1.amzn2023.x86_64
  mongodb-org-database-tools-extra-7.0.31-1.amzn2023.x86_64         mongodb-org-mongos-7.0.31-1.amzn2023.x86_64
  mongodb-org-server-7.0.31-1.amzn2023.x86_64                       mongodb-org-tools-7.0.31-1.amzn2023.x86_64

Complete!
db version v7.0.31
Build Info: {
    "version": "7.0.31",
    "gitVersion": "6a3bdfa2794c261d3ce011c74f818e970e563609",
    "openSSLVersion": "OpenSSL 3.5.5 27 Jan 2026",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "amazon2023",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
```

```
Installed:
  mongodb-database-tools-100.16.0-1.x86_64                          mongodb-mongosh-2.8.2-1.el8.x86_64
  mongodb-org-7.0.31-1.amzn2023.x86_64                              mongodb-org-database-7.0.31-1.amzn2023.x86_64
  mongodb-org-database-tools-extra-7.0.31-1.amzn2023.x86_64         mongodb-org-mongos-7.0.31-1.amzn2023.x86_64
  mongodb-org-server-7.0.31-1.amzn2023.x86_64                       mongodb-org-tools-7.0.31-1.amzn2023.x86_64

Complete!
db version v7.0.31
Build Info: {
    "version": "7.0.31",
    "gitVersion": "6a3bdfa2794c261d3ce011c74f818e970e563609",
    "openSSLVersion": "OpenSSL 3.5.5 27 Jan 2026",
    "modules": [],
    "allocator": "tcmalloc",
    "environment": {
        "distmod": "amazon2023",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}
[ec2-user@ip-172-31-77-233 ~]$
[ec2-user@ip-172-31-77-233 ~]$
[ec2-user@ip-172-31-77-233 ~]$
```

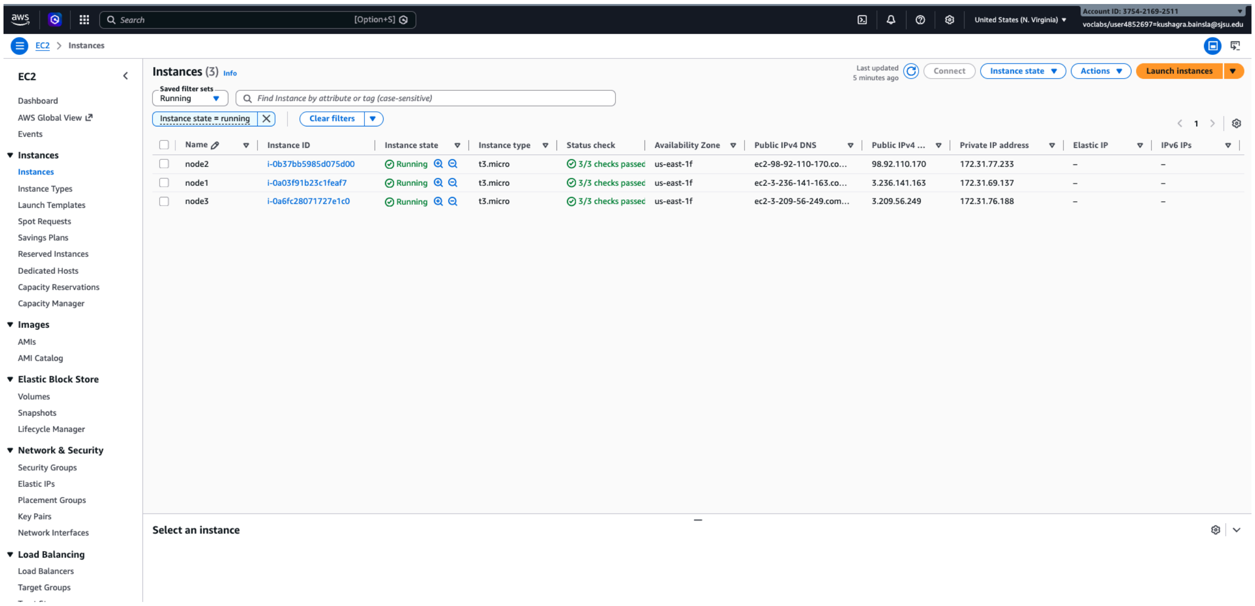
Task 4

```
ls -ld /data /data/configdb /data/shardA /data/shardB /data/shardC
drwxr-xr-x. 6 mongod mongod 64 Apr  8 00:55 /data
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/configdb
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardA
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardB
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardC
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
```

```
ls -ld /data /data/configdb /data/shardA /data/shardB /data/shardC
drwxr-xr-x. 6 mongod mongod 64 Apr  8 00:55 /data
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/configdb
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardA
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardB
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardC
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
[ec2-user@ip-172-31-76-188 ~]$
```

```
ls -ld /data /data/configdb /data/shardA /data/shardB /data/shardC
drwxr-xr-x. 6 mongod mongod 64 Apr  8 00:55 /data
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/configdb
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardA
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardB
drwxr-xr-x. 2 mongod mongod  6 Apr  8 00:55 /data/shardC
[ec2-user@ip-172-31-77-233 ~]$
[ec2-user@ip-172-31-77-233 ~]$
[ec2-user@ip-172-31-77-233 ~]$
```

Task 5



node1: public 3.236.141.163 | private 172.31.69.137
node2: public 98.92.110.170 | private 172.31.77.233
node3: public 3.209.56.249 | private 172.31.76.188

Task 6

```
test> rs.initiate({
|   _id: "csrs",
|   configsvr: true,
|   members: [
|     { _id: 0, host: "172.31.69.137:27019" },
|     { _id: 1, host: "172.31.77.233:27019" },
|     { _id: 2, host: "172.31.76.188:27019" }
|   ]
| })
{ ok: 1 }
csrs [direct: other] test> rs.status()
{
  set: 'csrs',
  date: ISODate('2026-04-08T01:07:02.280Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  configsvr: true,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775610421, i: 45 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:07:01.971Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775610421, i: 45 }),
t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775610421, i: 45 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775610421, i: 45 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:07:01.971Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:07:01.971Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775610410, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
```

```
    lastElectionDate: ISODate('2026-04-08T01:07:01.066Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775610410, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775610410, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:07:01.111Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:07:01.625Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27019',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 400,
      optime: { ts: Timestamp({ t: 1775610421, i: 45 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:07:01.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:07:01.971Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:07:01.971Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: 'Could not find member to sync from',
      electionTime: Timestamp({ t: 1775610421, i: 1 }),
      electionDate: ISODate('2026-04-08T01:07:01.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27019',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 11,
      optime: { ts: Timestamp({ t: 1775610410, i: 1 }), t: Long('-1') },
```

```
      optimeDurable: { ts: Timestamp({ t: 1775610410, i: 1 }), t: 
Long('-1') },
      optimeDate: ISODate('2026-04-08T01:06:50.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:06:50.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:07:01.919Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:07:01.919Z'),
      lastHeartbeat: ISODate('2026-04-08T01:07:01.079Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:07:02.081Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27019',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 11,
      optime: { ts: Timestamp({ t: 1775610410, i: 1 }), t: Long('-1') },
      optimeDurable: { ts: Timestamp({ t: 1775610410, i: 1 }), t: 
Long('-1') },
      optimeDate: ISODate('2026-04-08T01:06:50.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:06:50.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:07:01.971Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:07:01.971Z'),
      lastHeartbeat: ISODate('2026-04-08T01:07:01.078Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:07:02.078Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
```

```
    clusterTime: Timestamp({ t: 1775610421, i: 45 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775610421, i: 45 })
}
csrs [direct: primary] test>
```

Task 7

```
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27017
Current Mongosh Log ID: 69d5ab552ff91ff73544ba88
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:09:14.148+00:00: Access control is not enabled for the database. Read and write access to data and configurat
ion is unrestricted
------

[direct: mongos] test> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69d5aa35b5ce4cf680f3cbd9') }
---
shards
[]
---
active mongoses
[]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  }
]
[direct: mongos] test>
```

Task 8

Shard A

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27018
Current Mongosh Log ID: 69d5b418722c08c0dd44ba88
Connecting to:
mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:17:50.632+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardA [direct: primary] test> rs.status()
{
  set: 'shardA',
  date: ISODate('2026-04-08T01:49:18.202Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t:
Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
```

```
    durableOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775612944, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:20:24.121Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611213, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611213, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:20:24.171Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:20:24.678Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27018',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 1888,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1775611224, i: 1 }),
      electionDate: ISODate('2026-04-08T01:20:24.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
```

```
    {
      _id: 1,
      name: '172.31.77.233:27018',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1744,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastHeartbeat: ISODate('2026-04-08T01:49:16.217Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:49:17.158Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27018',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27018',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1744,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastHeartbeat: ISODate('2026-04-08T01:49:16.217Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:49:17.157Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27018',
```

```
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775612954, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775612954, i: 1 })
}
shardA [direct: primary] test>
```

Shard B

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27028
Current Mongosh Log ID: 69d5b4ab4d6dca108044ba88
Connecting to:
mongodb://127.0.0.1:27028/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:17:51.431+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardB [direct: primary] test> rs.status()
{
  set: 'shardB',
  date: ISODate('2026-04-08T01:51:46.744Z'),
```

```
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t:
Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775613053, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:30:13.080Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611801, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611801, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:30:13.126Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:30:13.632Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27028',
      health: 1,
```

```
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 2036,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1775611813, i: 1 }),
      electionDate: ISODate('2026-04-08T01:30:13.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27028',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1304,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastHeartbeat: ISODate('2026-04-08T01:51:45.185Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:51:46.136Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27028',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
```

```
      name: '172.31.76.188:27028',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1304,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastHeartbeat: ISODate('2026-04-08T01:51:45.185Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:51:46.124Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27028',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775613103, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775613103, i: 1 })
}
shardB [direct: primary] test>
```

Shard C

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27038
Current Mongosh Log ID: 69d5b4d09cc3a7090844ba88
Connecting to:
```

```
mongodb://127.0.0.1:27038/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:17:52.225+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardC [direct: primary] test> rs.status()
{
  set: 'shardC',
  date: ISODate('2026-04-08T01:52:20.726Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t:
Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775613122, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:33:22.638Z'),
```

```
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611991, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611991, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:33:22.681Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:33:23.189Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27038',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 2069,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1775612002, i: 1 }),
      electionDate: ISODate('2026-04-08T01:33:22.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27038',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1148,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
```

```
},
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastHeartbeat: ISODate('2026-04-08T01:52:20.697Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:52:19.691Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27038',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27038',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1148,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastHeartbeat: ISODate('2026-04-08T01:52:20.683Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:52:19.669Z'),
      pingMs: Long('1'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27038',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775613132, i: 1 }),
```

```
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775613132, i: 1 })
}
shardC [direct: primary] test>


Pre-add

[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27017
Current Mongosh Log ID: 69d5b524a875f65c5d44ba88
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:09:14.148+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

[direct: mongos] test> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69d5aa35b5ce4cf680f3cbd9') }
---
shards
[]
---
active mongoses
[]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
```

```
  'Currently running': 'no',
  'Currently enabled': 'yes',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  }
]
[direct: mongos] test>
```

Task 9

```
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27017
Current Mongosh Log ID: 69d5b5e293db2ffeb944ba88
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:09:14.148+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

[direct: mongos] test>
sh.addShard("shardA/172.31.69.137:27018,172.31.77.233:27018,172.31.76.188:2
7018")
|
sh.addShard("shardB/172.31.69.137:27028,172.31.77.233:27028,172.31.76.188:2
7028")
|
sh.addShard("shardC/172.31.69.137:27038,172.31.77.233:27038,172.31.76.188:2
7038")
|
| sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69d5aa35b5ce4cf680f3cbd9') }
---
shards
[
  {
    _id: 'shardA',
    host:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 2 })
```

```
  },
  {
    _id: 'shardB',
    host:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 6 })
  },
  {
    _id: 'shardC',
    host:
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 20 })
  }
]
---
active mongoses
[ { '7.0.31': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently running': 'no',
  'Currently enabled': 'yes',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  }
]
[direct: mongos] test>
```

Task 10

```
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27017
Current Mongosh Log ID: 69d5ba80ac5ddef9b844ba88
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:09:14.148+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

[direct: mongos] test> use midterm
switched to db midterm
[direct: mongos] midterm> sh.enableSharding("midterm")
| sh.shardCollection("midterm.records", { zip: "hashed" })
| sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69d5aa35b5ce4cf680f3cbd9') }
---
shards
[
  {
    _id: 'shardA',
    host:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 2 })
  },
  {
    _id: 'shardB',
    host:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
    state: 1,
```

```
    topologyTime: Timestamp({ t: 1775613417, i: 6 })
  },
  {
    _id: 'shardC',
    host:
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 20 })
  }
]
---
active mongoses
[ { '7.0.31': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Failed balancer rounds in last 5 attempts': 0,
  'Currently running': 'no',
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[
  {
    ns: 'midterm.records',
    shards: [
      {
        shardName: 'shardC',
        numOrphanedDocs: 0,
        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shardA',
        numOrphanedDocs: 0,
        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
```

```
      },
      {
        shardName: 'shardB',
        numOrphanedDocs: 0,
        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
      }
    ]
  },
  {
    ns: 'config.system.sessions',
    shards: [
      {
        shardName: 'shardA',
        numOrphanedDocs: 0,
        numOwnedDocuments: 45,
        ownedSizeBytes: 4455,
        orphanedSizeBytes: 0
      }
    ]
  }
]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {
      'config.system.sessions': {
        shardKey: { _id: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [ { shard: 'shardA', nChunks: 1 } ],
        chunks: [
          { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 
'shardA', 'last modified': Timestamp({ t: 1, i: 0 }) }
        ],
        tags: []
      }
    }
  },
```

```
  {
    database: {
      _id: 'midterm',
      primary: 'shardC',
      partitioned: false,
      version: {
        uuid: UUID('c039b27d-b65b-47ea-bba3-5da787424748'),
        timestamp: Timestamp({ t: 1775614606, i: 1 }),
        lastMod: 1
      }
    },
    collections: {
      'midterm.records': {
        shardKey: { zip: 'hashed' },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [
          { shard: 'shardA', nChunks: 2 },
          { shard: 'shardB', nChunks: 2 },
          { shard: 'shardC', nChunks: 2 }
        ],
        chunks: [
          { min: { zip: MinKey() }, max: { zip:
Long('-6148914691236517204') }, 'on shard': 'shardC', 'last modified':
Timestamp({ t: 1, i: 0 }) },
          { min: { zip: Long('-6148914691236517204') }, max: { zip:
Long('-3074457345618258602') }, 'on shard': 'shardC', 'last modified':
Timestamp({ t: 1, i: 1 }) },
          { min: { zip: Long('-3074457345618258602') }, max: { zip:
Long('0') }, 'on shard': 'shardA', 'last modified': Timestamp({ t: 1, i: 2
}) },
          { min: { zip: Long('0') }, max: { zip:
Long('3074457345618258602') }, 'on shard': 'shardA', 'last modified':
Timestamp({ t: 1, i: 3 }) },
          { min: { zip: Long('3074457345618258602') }, max: { zip:
Long('6148914691236517204') }, 'on shard': 'shardB', 'last modified':
Timestamp({ t: 1, i: 4 }) },
          { min: { zip: Long('6148914691236517204') }, max: { zip: MaxKey()
}, 'on shard': 'shardB', 'last modified': Timestamp({ t: 1, i: 5 }) }
        ],
        tags: []
      }
```

```
    }
  }
]
[direct: mongos] midterm>
```

Database: midterm
Collection: records
Shard key: zip (hashed)

Shard key nature:
- The key is zip, which becomes effectively random when hashed.
- Hashed keys spread values uniformly across chunks and shards.

Sharding strategy:
- Hash-based sharding on { zip: "hashed" }.
- This improves distribution and balances data across all shards.

Commands used:
- sh.enableSharding("midterm")
- sh.shardCollection("midterm.records", { zip: "hashed" })

Task 11

```
[ec2-user@ip-172-31-69-137 data]$ mongoimport --port 27017 --db midterm
--collection records --file 311_with_zip.json --jsonArray
2026-04-08T03:24:03.395+0000    connected to: mongodb://localhost:27017/
2026-04-08T03:24:06.396+0000    [#######################.] midterm.records
10.4MB/10.6MB (98.4%)
2026-04-08T03:24:06.459+0000    [########################] midterm.records
10.6MB/10.6MB (100.0%)
2026-04-08T03:24:06.460+0000    50000 document(s) imported successfully. 0
document(s) failed to import.
[ec2-user@ip-172-31-69-137 data]$ mongosh --port 27017
Current Mongosh Log ID: 69d5ca598c978012d544ba88
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:09:14.148+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

[direct: mongos] test> use midterm
switched to db midterm
[direct: mongos] midterm> db.records.countDocuments()
100000
[direct: mongos] midterm> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69d5aa35b5ce4cf680f3cbd9') }
---
shards
[
  {
    _id: 'shardA',
    host:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
    state: 1,
```

```
    topologyTime: Timestamp({ t: 1775613417, i: 2 })
  },
  {
    _id: 'shardB',
    host:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 6 })
  },
  {
    _id: 'shardC',
    host:
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
    state: 1,
    topologyTime: Timestamp({ t: 1775613417, i: 20 })
  }
]
---
active mongoses
[ { '7.0.31': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[
  {
    ns: 'config.system.sessions',
    shards: [
      {
        shardName: 'shardA',
        numOrphanedDocs: 0,
        numOwnedDocuments: 9,
        ownedSizeBytes: 891,
        orphanedSizeBytes: 0
```

```
      }
    ]
  },
  {
    ns: 'midterm.records',
    shards: [
      {
        shardName: 'shardB',
        numOrphanedDocs: 0,
        numOwnedDocuments: 36881,
        ownedSizeBytes: 8777678,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shardA',
        numOrphanedDocs: 0,
        numOwnedDocuments: 33361,
        ownedSizeBytes: 7939918,
        orphanedSizeBytes: 0
      },
      {
        shardName: 'shardC',
        numOrphanedDocs: 0,
        numOwnedDocuments: 29758,
        ownedSizeBytes: 7082404,
        orphanedSizeBytes: 0
      }
    ]
  }
]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {
      'config.system.sessions': {
        shardKey: { _id: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [ { shard: 'shardA', nChunks: 1 } ],
        chunks: [
```

```
          { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard':
'shardA', 'last modified': Timestamp({ t: 1, i: 0 }) }
        ],
        tags: []
      }
    }
  },
  {
    database: {
      _id: 'midterm',
      primary: 'shardC',
      partitioned: false,
      version: {
        uuid: UUID('c039b27d-b65b-47ea-bba3-5da787424748'),
        timestamp: Timestamp({ t: 1775614606, i: 1 }),
        lastMod: 1
      }
    },
    collections: {
      'midterm.records': {
        shardKey: { zip: 'hashed' },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [
          { shard: 'shardA', nChunks: 1 },
          { shard: 'shardB', nChunks: 1 },
          { shard: 'shardC', nChunks: 1 }
        ],
        chunks: [
          { min: { zip: MinKey() }, max: { zip:
Long('-3074457345618258602') }, 'on shard': 'shardC', 'last modified':
Timestamp({ t: 1, i: 8 }) },
          { min: { zip: Long('-3074457345618258602') }, max: { zip:
Long('3074457345618258602') }, 'on shard': 'shardA', 'last modified':
Timestamp({ t: 1, i: 6 }) },
          { min: { zip: Long('3074457345618258602') }, max: { zip: MaxKey()
}, 'on shard': 'shardB', 'last modified': Timestamp({ t: 1, i: 7 }) }
        ],
        tags: []
      }
    }
  }
```

```
]
[direct: mongos] midterm>
```

Dataset:
- NYC 311 Service Requests (public)
- URL: https://data.cityofnewyork.us/resource/erm2-nwe9.json

Collection and fields:
- Database: midterm
- Collection: records
- Added field: zip (copied from incident_zip) to match shard key

Load process summary:
1) Downloaded a filtered JSON subset (50,000 records) with fields:
   unique_key, incident_zip, agency, complaint_type, descriptor, created_date, borough
2) Added zip = incident_zip for shard key compatibility.
3) Imported JSON into midterm.records via mongoimport.
4) The dataset was imported twice during retries to increase size for shard distribution (final count 100,000).
5) Verified distribution with sh.status() and record count.

Commands used (summary):
- curl ... erm2-nwe9.json ... -o 311_raw.json
- python3: add zip field, write 311_with_zip.json
- mongoimport --port 27017 --db midterm --collection records --file 311_with_zip.json --jsonArray
- db.records.countDocuments()
- sh.status()

Task 12

1) Range query note:
- The shard key is { zip: "hashed" }, so range queries on zipNum cannot be targeted to a single shard.
- This is why the range query scans all shards.

2) $elemMatch note:
- tags is an array of key-value objects (e.g., { k: "borough", v: "BROOKLYN" }).
- $elemMatch is used to match multiple conditions within the same array element.

Query 1

```
[direct: mongos] midterm> db.records.find({ zipNum: { $gte: 10000, $lte:
20000 } }).explain("executionStats")
{
  queryPlanner: {
    mongosPlannerVersion: 1,
    winningPlan: {
      stage: 'SHARD_MERGE',
      shards: [
        {
          shardName: 'shardC',
          connectionString:
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
          serverInfo: {
            host: 'ip-172-31-76-188.ec2.internal',
            port: 27038,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: {
            '$and': [
              { zipNum: { '$lte': 20000 } },
              { zipNum: { '$gte': 10000 } }
            ]
```

```
          },
          queryHash: '4103EB4C',
          planCacheKey: '4103EB4C',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                '$and': [
                  { zipNum: { '$lte': 20000 } },
                  { zipNum: { '$gte': 10000 } }
                ]
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        },
        {
          shardName: 'shardA',
          connectionString:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
          serverInfo: {
            host: 'ip-172-31-77-233.ec2.internal',
            port: 27018,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: {
            '$and': [
              { zipNum: { '$lte': 20000 } },
              { zipNum: { '$gte': 10000 } }
            ]
          },
          queryHash: '4103EB4C',
          planCacheKey: '4103EB4C',
          optimizationTimeMillis: 0,
```

```
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                '$and': [
                  { zipNum: { '$lte': 20000 } },
                  { zipNum: { '$gte': 10000 } }
                ]
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        },
        {
          shardName: 'shardB',
          connectionString:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
          serverInfo: {
            host: 'ip-172-31-77-233.ec2.internal',
            port: 27028,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: {
            '$and': [
              { zipNum: { '$lte': 20000 } },
              { zipNum: { '$gte': 10000 } }
            ]
          },
          queryHash: '4103EB4C',
          planCacheKey: '4103EB4C',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
```

```
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                '$and': [
                  { zipNum: { '$lte': 20000 } },
                  { zipNum: { '$gte': 10000 } }
                ]
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        }
      ]
    }
  },
  executionStats: {
    nReturned: 99699,
    executionTimeMillis: 110,
    totalKeysExamined: 0,
    totalDocsExamined: 99699,
    executionStages: {
      stage: 'SHARD_MERGE',
      nReturned: 99699,
      executionTimeMillis: 110,
      totalKeysExamined: 0,
      totalDocsExamined: 99699,
      totalChildMillis: Long('256'),
      shards: [
        {
          shardName: 'shardC',
          executionSuccess: true,
          nReturned: 29642,
          executionTimeMillis: 51,
          totalKeysExamined: 0,
          totalDocsExamined: 29642,
          executionStages: {
            stage: 'SHARDING_FILTER',
            nReturned: 29642,
            executionTimeMillisEstimate: 8,
            works: 29643,
            advanced: 29642,
```

```
        needTime: 0,
        needYield: 0,
        saveState: 29,
        restoreState: 29,
        isEOF: 1,
        chunkSkips: 0,
        inputStage: {
          stage: 'COLLSCAN',
          filter: {
            '$and': [
              { zipNum: { '$lte': 20000 } },
              { zipNum: { '$gte': 10000 } }
            ]
          },
          nReturned: 29642,
          executionTimeMillisEstimate: 7,
          works: 29643,
          advanced: 29642,
          needTime: 0,
          needYield: 0,
          saveState: 29,
          restoreState: 29,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 29642
        }
      }
    },
    {
      shardName: 'shardA',
      executionSuccess: true,
      nReturned: 33273,
      executionTimeMillis: 101,
      totalKeysExamined: 0,
      totalDocsExamined: 33273,
      executionStages: {
        stage: 'SHARDING_FILTER',
        nReturned: 33273,
        executionTimeMillisEstimate: 21,
        works: 33274,
        advanced: 33273,
        needTime: 0,
        needYield: 0,
```

```
        saveState: 33,
        restoreState: 33,
        isEOF: 1,
        chunkSkips: 0,
        inputStage: {
          stage: 'COLLSCAN',
          filter: {
            '$and': [
              { zipNum: { '$lte': 20000 } },
              { zipNum: { '$gte': 10000 } }
            ]
          },
          nReturned: 33273,
          executionTimeMillisEstimate: 9,
          works: 33274,
          advanced: 33273,
          needTime: 0,
          needYield: 0,
          saveState: 33,
          restoreState: 33,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 33273
        }
      }
    },
    {
      shardName: 'shardB',
      executionSuccess: true,
      nReturned: 36784,
      executionTimeMillis: 104,
      totalKeysExamined: 0,
      totalDocsExamined: 36784,
      executionStages: {
        stage: 'SHARDING_FILTER',
        nReturned: 36784,
        executionTimeMillisEstimate: 21,
        works: 36785,
        advanced: 36784,
        needTime: 0,
        needYield: 0,
        saveState: 36,
        restoreState: 36,
```

```
          isEOF: 1,
          chunkSkips: 0,
          inputStage: {
            stage: 'COLLSCAN',
            filter: {
              '$and': [
                { zipNum: { '$lte': 20000 } },
                { zipNum: { '$gte': 10000 } }
              ]
            },
            nReturned: 36784,
            executionTimeMillisEstimate: 16,
            works: 36785,
            advanced: 36784,
            needTime: 0,
            needYield: 0,
            saveState: 36,
            restoreState: 36,
            isEOF: 1,
            direction: 'forward',
            docsExamined: 36784
          }
        }
      }
    ]
  }
},
serverInfo: {
  host: 'ip-172-31-69-137.ec2.internal',
  port: 27017,
  version: '7.0.31',
  gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'forceClassicEngine'
```

```
  },
  command: {
    find: 'records',
    filter: { zipNum: { '$gte': 10000, '$lte': 20000 } },
    lsid: { id: UUID('dc086896-ad3b-483a-a397-6dbe15628238') },
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1775619118, i: 1 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: 0
      }
    },
    '$db': 'midterm'
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775619130, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775619126, i: 1 })
}
[direct: mongos] midterm>
```

Query 2

```
[direct: mongos] midterm> db.records.find({
|   tags: { $elemMatch: { k: "borough", v: "BROOKLYN" } }
| }).explain("executionStats")
{
  queryPlanner: {
    mongosPlannerVersion: 1,
    winningPlan: {
      stage: 'SHARD_MERGE',
      shards: [
        {
          shardName: 'shardC',
          connectionString:
```

```
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
          serverInfo: {
            host: 'ip-172-31-76-188.ec2.internal',
            port: 27038,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: {
            tags: {
              '$elemMatch': {
                '$and': [
                  { k: { '$eq': 'borough' } },
                  { v: { '$eq': 'BROOKLYN' } }
                ]
              }
            }
          },
          queryHash: '4787770D',
          planCacheKey: '4787770D',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                tags: {
                  '$elemMatch': {
                    '$and': [
                      { k: { '$eq': 'borough' } },
                      { v: { '$eq': 'BROOKLYN' } }
                    ]
                  }
                }
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
```

```
        },
        {
          shardName: 'shardB',
          connectionString:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
          serverInfo: {
            host: 'ip-172-31-77-233.ec2.internal',
            port: 27028,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: {
            tags: {
              '$elemMatch': {
                '$and': [
                  { k: { '$eq': 'borough' } },
                  { v: { '$eq': 'BROOKLYN' } }
                ]
              }
            }
          },
          queryHash: '4787770D',
          planCacheKey: '4787770D',
          optimizationTimeMillis: 1,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                tags: {
                  '$elemMatch': {
                    '$and': [
                      { k: { '$eq': 'borough' } },
                      { v: { '$eq': 'BROOKLYN' } }
                    ]
                  }
                }
              },
```

```
            direction: 'forward'
          }
        },
        rejectedPlans: []
      },
      {
        shardName: 'shardA',
        connectionString:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
        serverInfo: {
          host: 'ip-172-31-77-233.ec2.internal',
          port: 27018,
          version: '7.0.31',
          gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
        },
        namespace: 'midterm.records',
        indexFilterSet: false,
        parsedQuery: {
          tags: {
            '$elemMatch': {
              '$and': [
                { k: { '$eq': 'borough' } },
                { v: { '$eq': 'BROOKLYN' } }
              ]
            }
          }
        },
        queryHash: '4787770D',
        planCacheKey: '4787770D',
        optimizationTimeMillis: 1,
        maxIndexedOrSolutionsReached: false,
        maxIndexedAndSolutionsReached: false,
        maxScansToExplodeReached: false,
        winningPlan: {
          stage: 'SHARDING_FILTER',
          inputStage: {
            stage: 'COLLSCAN',
            filter: {
              tags: {
                '$elemMatch': {
                  '$and': [
                    { k: { '$eq': 'borough' } },
                    { v: { '$eq': 'BROOKLYN' } }
```

```
                    ]
                  }
                }
              },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        }
      ]
    }
  },
  executionStats: {
    nReturned: 29541,
    executionTimeMillis: 91,
    totalKeysExamined: 0,
    totalDocsExamined: 99699,
    executionStages: {
      stage: 'SHARD_MERGE',
      nReturned: 29541,
      executionTimeMillis: 91,
      totalKeysExamined: 0,
      totalDocsExamined: 99699,
      totalChildMillis: Long('195'),
      shards: [
        {
          shardName: 'shardC',
          executionSuccess: true,
          nReturned: 11541,
          executionTimeMillis: 35,
          totalKeysExamined: 0,
          totalDocsExamined: 29642,
          executionStages: {
            stage: 'SHARDING_FILTER',
            nReturned: 11541,
            executionTimeMillisEstimate: 6,
            works: 29643,
            advanced: 11541,
            needTime: 18101,
            needYield: 0,
            saveState: 29,
            restoreState: 29,
            isEOF: 1,
```

```
          chunkSkips: 0,
          inputStage: {
            stage: 'COLLSCAN',
            filter: {
              tags: {
                '$elemMatch': {
                  '$and': [
                    { k: { '$eq': 'borough' } },
                    { v: { '$eq': 'BROOKLYN' } }
                  ]
                }
              }
            },
            nReturned: 11541,
            executionTimeMillisEstimate: 4,
            works: 29643,
            advanced: 11541,
            needTime: 18101,
            needYield: 0,
            saveState: 29,
            restoreState: 29,
            isEOF: 1,
            direction: 'forward',
            docsExamined: 29642
          }
        }
      },
      {
        shardName: 'shardB',
        executionSuccess: true,
        nReturned: 8377,
        executionTimeMillis: 81,
        totalKeysExamined: 0,
        totalDocsExamined: 36784,
        executionStages: {
          stage: 'SHARDING_FILTER',
          nReturned: 8377,
          executionTimeMillisEstimate: 17,
          works: 36785,
          advanced: 8377,
          needTime: 28407,
          needYield: 0,
          saveState: 36,
```

```
        restoreState: 36,
        isEOF: 1,
        chunkSkips: 0,
        inputStage: {
          stage: 'COLLSCAN',
          filter: {
            tags: {
              '$elemMatch': {
                '$and': [
                  { k: { '$eq': 'borough' } },
                  { v: { '$eq': 'BROOKLYN' } }
                ]
              }
            }
          },
          nReturned: 8377,
          executionTimeMillisEstimate: 17,
          works: 36785,
          advanced: 8377,
          needTime: 28407,
          needYield: 0,
          saveState: 36,
          restoreState: 36,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 36784
        }
      }
    },
    {
      shardName: 'shardA',
      executionSuccess: true,
      nReturned: 9623,
      executionTimeMillis: 79,
      totalKeysExamined: 0,
      totalDocsExamined: 33273,
      executionStages: {
        stage: 'SHARDING_FILTER',
        nReturned: 9623,
        executionTimeMillisEstimate: 17,
        works: 33274,
        advanced: 9623,
        needTime: 23650,
```

```
            needYield: 0,
            saveState: 33,
            restoreState: 33,
            isEOF: 1,
            chunkSkips: 0,
            inputStage: {
              stage: 'COLLSCAN',
              filter: {
                tags: {
                  '$elemMatch': {
                    '$and': [
                      { k: { '$eq': 'borough' } },
                      { v: { '$eq': 'BROOKLYN' } }
                    ]
                  }
                }
              },
              nReturned: 9623,
              executionTimeMillisEstimate: 16,
              works: 33274,
              advanced: 9623,
              needTime: 23650,
              needYield: 0,
              saveState: 33,
              restoreState: 33,
              isEOF: 1,
              direction: 'forward',
              docsExamined: 33273
            }
          }
        }
      ]
    }
  },
  serverInfo: {
    host: 'ip-172-31-69-137.ec2.internal',
    port: 27017,
    version: '7.0.31',
    gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
```

```
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  command: {
    find: 'records',
    filter: { tags: { '$elemMatch': { k: 'borough', v: 'BROOKLYN' } } },
    lsid: { id: UUID('dc086896-ad3b-483a-a397-6dbe15628238') },
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1775619344, i: 2 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: 0
      }
    },
    '$db': 'midterm'
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775619450, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775619446, i: 1 })
}
[direct: mongos] midterm>
```

Query 3

```
[direct: mongos] midterm> db.records.find({
|   borough: { $in: ["BROOKLYN", "QUEENS"] }
| }).explain("executionStats")
{
  queryPlanner: {
```

```
    mongosPlannerVersion: 1,
    winningPlan: {
      stage: 'SHARD_MERGE',
      shards: [
        {
          shardName: 'shardC',
          connectionString:
'shardC/172.31.69.137:27038,172.31.76.188:27038,172.31.77.233:27038',
          serverInfo: {
            host: 'ip-172-31-76-188.ec2.internal',
            port: 27038,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
          queryHash: '0E89A6B3',
          planCacheKey: '0E89A6B3',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        },
        {
          shardName: 'shardA',
          connectionString:
'shardA/172.31.69.137:27018,172.31.76.188:27018,172.31.77.233:27018',
          serverInfo: {
            host: 'ip-172-31-77-233.ec2.internal',
            port: 27018,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
```

```
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
          queryHash: '0E89A6B3',
          planCacheKey: '0E89A6B3',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
              direction: 'forward'
            }
          },
          rejectedPlans: []
        },
        {
          shardName: 'shardB',
          connectionString:
'shardB/172.31.69.137:27028,172.31.76.188:27028,172.31.77.233:27028',
          serverInfo: {
            host: 'ip-172-31-77-233.ec2.internal',
            port: 27028,
            version: '7.0.31',
            gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
          },
          namespace: 'midterm.records',
          indexFilterSet: false,
          parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
          queryHash: '0E89A6B3',
          planCacheKey: '0E89A6B3',
          optimizationTimeMillis: 0,
          maxIndexedOrSolutionsReached: false,
          maxIndexedAndSolutionsReached: false,
          maxScansToExplodeReached: false,
          winningPlan: {
            stage: 'SHARDING_FILTER',
            inputStage: {
              stage: 'COLLSCAN',
              filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
```

```
              direction: 'forward'
            }
          },
          rejectedPlans: []
        }
      ]
    }
  },
  executionStats: {
    nReturned: 54364,
    executionTimeMillis: 67,
    totalKeysExamined: 0,
    totalDocsExamined: 99699,
    executionStages: {
      stage: 'SHARD_MERGE',
      nReturned: 54364,
      executionTimeMillis: 67,
      totalKeysExamined: 0,
      totalDocsExamined: 99699,
      totalChildMillis: Long('162'),
      shards: [
        {
          shardName: 'shardC',
          executionSuccess: true,
          nReturned: 19417,
          executionTimeMillis: 32,
          totalKeysExamined: 0,
          totalDocsExamined: 29642,
          executionStages: {
            stage: 'SHARDING_FILTER',
            nReturned: 19417,
            executionTimeMillisEstimate: 6,
            works: 29643,
            advanced: 19417,
            needTime: 10225,
            needYield: 0,
            saveState: 29,
            restoreState: 29,
            isEOF: 1,
            chunkSkips: 0,
            inputStage: {
              stage: 'COLLSCAN',
              filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
```

```
          nReturned: 19417,
          executionTimeMillisEstimate: 4,
          works: 29643,
          advanced: 19417,
          needTime: 10225,
          needYield: 0,
          saveState: 29,
          restoreState: 29,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 29642
        }
      }
    },
    {
      shardName: 'shardA',
      executionSuccess: true,
      nReturned: 18071,
      executionTimeMillis: 64,
      totalKeysExamined: 0,
      totalDocsExamined: 33273,
      executionStages: {
        stage: 'SHARDING_FILTER',
        nReturned: 18071,
        executionTimeMillisEstimate: 7,
        works: 33274,
        advanced: 18071,
        needTime: 15202,
        needYield: 0,
        saveState: 33,
        restoreState: 33,
        isEOF: 1,
        chunkSkips: 0,
        inputStage: {
          stage: 'COLLSCAN',
          filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
          nReturned: 18071,
          executionTimeMillisEstimate: 3,
          works: 33274,
          advanced: 18071,
          needTime: 15202,
          needYield: 0,
          saveState: 33,
```

```
          restoreState: 33,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 33273
        }
      }
    },
    {
      shardName: 'shardB',
      executionSuccess: true,
      nReturned: 16876,
      executionTimeMillis: 66,
      totalKeysExamined: 0,
      totalDocsExamined: 36784,
      executionStages: {
        stage: 'SHARDING_FILTER',
        nReturned: 16876,
        executionTimeMillisEstimate: 8,
        works: 36785,
        advanced: 16876,
        needTime: 19908,
        needYield: 0,
        saveState: 36,
        restoreState: 36,
        isEOF: 1,
        chunkSkips: 0,
        inputStage: {
          stage: 'COLLSCAN',
          filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
          nReturned: 16876,
          executionTimeMillisEstimate: 3,
          works: 36785,
          advanced: 16876,
          needTime: 19908,
          needYield: 0,
          saveState: 36,
          restoreState: 36,
          isEOF: 1,
          direction: 'forward',
          docsExamined: 36784
        }
      }
    }
```

```
      ]
    }
  },
  serverInfo: {
    host: 'ip-172-31-69-137.ec2.internal',
    port: 27017,
    version: '7.0.31',
    gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  command: {
    find: 'records',
    filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } },
    lsid: { id: UUID('dc086896-ad3b-483a-a397-6dbe15628238') },
    '$clusterTime': {
      clusterTime: Timestamp({ t: 1775619190, i: 1 }),
      signature: {
        hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
        keyId: 0
      }
    },
    '$db': 'midterm'
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775619253, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775619246, i: 1 })
}
```

```
[direct: mongos] midterm>
```

Query 4

```
[direct: mongos] midterm> db.records.aggregate([
|   { $match: { borough: { $in: ["BROOKLYN", "QUEENS"] } } },
|   { $group: { _id: "$borough", count: { $sum: 1 } } },
|   { $sort: { count: -1 } }
| ]).explain("executionStats")
{
  serverInfo: {
    host: 'ip-172-31-69-137.ec2.internal',
    port: 27017,
    version: '7.0.31',
    gitVersion: '6a3bdfa2794c261d3ce011c74f818e970e563609'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'forceClassicEngine'
  },
  mergeType: 'mongos',
  splitPipeline: {
    shardsPart: [
      {
        '$match': { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } }
      },
      {
        '$group': { _id: '$borough', count: { '$sum': { '$const': 1 } } }
      }
    ],
    mergerPart: [
      {
        '$mergeCursors': {
```

```
          lsid: {
            id: UUID('dc086896-ad3b-483a-a397-6dbe15628238'),
            uid:
Binary.createFromBase64('47DEQpj8HBSa+/TImW+5JCeuQeRkm5NMpJWZG3hSuFU=', 0)
          },
          compareWholeSortKey: false,
          tailableMode: 'normal',
          nss: 'midterm.records',
          allowPartialResults: false,
          recordRemoteOpWaitTime: false
        }
      },
      {
        '$group': {
          _id: '$$ROOT._id',
          count: { '$sum': '$$ROOT.count' },
          '$doingMerge': true
        }
      },
      { '$sort': { sortKey: { count: -1 } } }
    ]
  },
  shards: {
    shardC: {
      host: '172.31.76.188:27038',
      stages: [
        {
          '$cursor': {
            queryPlanner: {
              namespace: 'midterm.records',
              indexFilterSet: false,
              parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
              queryHash: '869FB6CE',
              planCacheKey: '869FB6CE',
              optimizationTimeMillis: 0,
              maxIndexedOrSolutionsReached: false,
              maxIndexedAndSolutionsReached: false,
              maxScansToExplodeReached: false,
              winningPlan: {
                stage: 'PROJECTION_SIMPLE',
                transformBy: { borough: 1, _id: 0 },
                inputStage: {
```

```
                  stage: 'SHARDING_FILTER',
                  inputStage: {
                    stage: 'COLLSCAN',
                    filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
                    direction: 'forward'
                  }
                }
              },
              rejectedPlans: []
            },
            executionStats: {
              executionSuccess: true,
              nReturned: 19417,
              executionTimeMillis: 56,
              totalKeysExamined: 0,
              totalDocsExamined: 29642,
              executionStages: {
                stage: 'PROJECTION_SIMPLE',
                nReturned: 19417,
                executionTimeMillisEstimate: 8,
                works: 29643,
                advanced: 19417,
                needTime: 10225,
                needYield: 0,
                saveState: 30,
                restoreState: 30,
                isEOF: 1,
                transformBy: { borough: 1, _id: 0 },
                inputStage: {
                  stage: 'SHARDING_FILTER',
                  nReturned: 19417,
                  executionTimeMillisEstimate: 8,
                  works: 29643,
                  advanced: 19417,
                  needTime: 10225,
                  needYield: 0,
                  saveState: 30,
                  restoreState: 30,
                  isEOF: 1,
                  chunkSkips: 0,
                  inputStage: {
                    stage: 'COLLSCAN',
```

```
                    filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
                    nReturned: 19417,
                    executionTimeMillisEstimate: 7,
                    works: 29643,
                    advanced: 19417,
                    needTime: 10225,
                    needYield: 0,
                    saveState: 30,
                    restoreState: 30,
                    isEOF: 1,
                    direction: 'forward',
                    docsExamined: 29642
                  }
                }
              }
            }
          },
          nReturned: Long('19417'),
          executionTimeMillisEstimate: Long('41')
        },
        {
          '$group': { _id: '$borough', count: { '$sum': { '$const': 1 } }
},
          maxAccumulatorMemoryUsageBytes: { count: Long('192') },
          totalOutputDataSizeBytes: Long('666'),
          usedDisk: false,
          spills: Long('0'),
          spilledDataStorageSize: Long('0'),
          numBytesSpilledEstimate: Long('0'),
          spilledRecords: Long('0'),
          nReturned: Long('2'),
          executionTimeMillisEstimate: Long('41')
        }
      ]
    },
    shardB: {
      host: '172.31.77.233:27028',
      stages: [
        {
          '$cursor': {
            queryPlanner: {
              namespace: 'midterm.records',
```

```
            indexFilterSet: false,
            parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
            queryHash: '869FB6CE',
            planCacheKey: '869FB6CE',
            optimizationTimeMillis: 0,
            maxIndexedOrSolutionsReached: false,
            maxIndexedAndSolutionsReached: false,
            maxScansToExplodeReached: false,
            winningPlan: {
              stage: 'PROJECTION_SIMPLE',
              transformBy: { borough: 1, _id: 0 },
              inputStage: {
                stage: 'SHARDING_FILTER',
                inputStage: {
                  stage: 'COLLSCAN',
                  filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
                  direction: 'forward'
                }
              }
            },
            rejectedPlans: []
          },
          executionStats: {
            executionSuccess: true,
            nReturned: 16876,
            executionTimeMillis: 92,
            totalKeysExamined: 0,
            totalDocsExamined: 36784,
            executionStages: {
              stage: 'PROJECTION_SIMPLE',
              nReturned: 16876,
              executionTimeMillisEstimate: 13,
              works: 36785,
              advanced: 16876,
              needTime: 19908,
              needYield: 0,
              saveState: 37,
              restoreState: 37,
              isEOF: 1,
              transformBy: { borough: 1, _id: 0 },
              inputStage: {
```

```
              stage: 'SHARDING_FILTER',
              nReturned: 16876,
              executionTimeMillisEstimate: 11,
              works: 36785,
              advanced: 16876,
              needTime: 19908,
              needYield: 0,
              saveState: 37,
              restoreState: 37,
              isEOF: 1,
              chunkSkips: 0,
              inputStage: {
                stage: 'COLLSCAN',
                filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
                nReturned: 16876,
                executionTimeMillisEstimate: 9,
                works: 36785,
                advanced: 16876,
                needTime: 19908,
                needYield: 0,
                saveState: 37,
                restoreState: 37,
                isEOF: 1,
                direction: 'forward',
                docsExamined: 36784
              }
            }
          }
        }
      },
      nReturned: Long('16876'),
      executionTimeMillisEstimate: Long('79')
    },
    {
      '$group': { _id: '$borough', count: { '$sum': { '$const': 1 } }
},
      maxAccumulatorMemoryUsageBytes: { count: Long('192') },
      totalOutputDataSizeBytes: Long('666'),
      usedDisk: false,
      spills: Long('0'),
      spilledDataStorageSize: Long('0'),
      numBytesSpilledEstimate: Long('0'),
```

```
          spilledRecords: Long('0'),
          nReturned: Long('2'),
          executionTimeMillisEstimate: Long('89')
        }
      ]
    },
    shardA: {
      host: '172.31.77.233:27018',
      stages: [
        {
          '$cursor': {
            queryPlanner: {
              namespace: 'midterm.records',
              indexFilterSet: false,
              parsedQuery: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
              queryHash: '869FB6CE',
              planCacheKey: '869FB6CE',
              optimizationTimeMillis: 0,
              maxIndexedOrSolutionsReached: false,
              maxIndexedAndSolutionsReached: false,
              maxScansToExplodeReached: false,
              winningPlan: {
                stage: 'PROJECTION_SIMPLE',
                transformBy: { borough: 1, _id: 0 },
                inputStage: {
                  stage: 'SHARDING_FILTER',
                  inputStage: {
                    stage: 'COLLSCAN',
                    filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
                    direction: 'forward'
                  }
                }
              },
              rejectedPlans: []
            },
            executionStats: {
              executionSuccess: true,
              nReturned: 18071,
              executionTimeMillis: 101,
              totalKeysExamined: 0,
              totalDocsExamined: 33273,
```

```
        executionStages: {
          stage: 'PROJECTION_SIMPLE',
          nReturned: 18071,
          executionTimeMillisEstimate: 30,
          works: 33274,
          advanced: 18071,
          needTime: 15202,
          needYield: 0,
          saveState: 34,
          restoreState: 34,
          isEOF: 1,
          transformBy: { borough: 1, _id: 0 },
          inputStage: {
            stage: 'SHARDING_FILTER',
            nReturned: 18071,
            executionTimeMillisEstimate: 21,
            works: 33274,
            advanced: 18071,
            needTime: 15202,
            needYield: 0,
            saveState: 34,
            restoreState: 34,
            isEOF: 1,
            chunkSkips: 0,
            inputStage: {
              stage: 'COLLSCAN',
              filter: { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] }
},
              nReturned: 18071,
              executionTimeMillisEstimate: 6,
              works: 33274,
              advanced: 18071,
              needTime: 15202,
              needYield: 0,
              saveState: 34,
              restoreState: 34,
              isEOF: 1,
              direction: 'forward',
              docsExamined: 33273
            }
          }
        }
      }
```

```
          },
          nReturned: Long('18071'),
          executionTimeMillisEstimate: Long('93')
        },
        {
          '$group': { _id: '$borough', count: { '$sum': { '$const': 1 } }
},
          maxAccumulatorMemoryUsageBytes: { count: Long('192') },
          totalOutputDataSizeBytes: Long('666'),
          usedDisk: false,
          spills: Long('0'),
          spilledDataStorageSize: Long('0'),
          numBytesSpilledEstimate: Long('0'),
          spilledRecords: Long('0'),
          nReturned: Long('2'),
          executionTimeMillisEstimate: Long('98')
        }
      ]
    }
  },
  command: {
    aggregate: 'records',
    pipeline: [
      {
        '$match': { borough: { '$in': [ 'BROOKLYN', 'QUEENS' ] } }
      },
      { '$group': { _id: '$borough', count: { '$sum': 1 } } },
      { '$sort': { count: -1 } }
    ],
    cursor: {}
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775619284, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775619284, i: 1 })
}
[direct: mongos] midterm>
```

Query 5

```
[direct: mongos] midterm> db.records.updateMany(
|   { borough: "MANHATTAN", flag2: { $exists: false } },
|   { $set: { flag2: "manhattan2" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19936,
  modifiedCount: 19936,
  upsertedCount: 0
}
```

Query 6

```
[direct: mongos] midterm> db.records.deleteMany({ complaint_type: "Noise -
Residential" })
{ acknowledged: true, deletedCount: 14431 }
[direct: mongos] midterm>
```

Task 13

Shard A

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27018
Current Mongosh Log ID: 69d5b418722c08c0dd44ba88
Connecting to:
mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:17:50.632+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardA [direct: primary] test> rs.status()
{
  set: 'shardA',
  date: ISODate('2026-04-08T01:49:18.202Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
```

```
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t:
Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775612944, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:20:24.121Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611213, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611213, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:20:24.171Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:20:24.678Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27018',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 1888,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      syncSourceHost: '',
      syncSourceId: -1,
```

```
      infoMessage: '',
      electionTime: Timestamp({ t: 1775611224, i: 1 }),
      electionDate: ISODate('2026-04-08T01:20:24.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27018',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1744,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastHeartbeat: ISODate('2026-04-08T01:49:16.217Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:49:17.158Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27018',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27018',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1744,
      optime: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775612954, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:49:14.000Z'),
```

```
      optimeDurableDate: ISODate('2026-04-08T01:49:14.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:49:14.315Z'),
      lastHeartbeat: ISODate('2026-04-08T01:49:16.217Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:49:17.157Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27018',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775612954, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775612954, i: 1 })
}
shardA [direct: primary] test>
```

Shard B

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27028
Current Mongosh Log ID: 69d5b4ab4d6dca108044ba88
Connecting to:
mongodb://127.0.0.1:27028/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
```

```
   The server generated these startup warnings when booting
   2026-04-08T01:17:51.431+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardB [direct: primary] test> rs.status()
{
  set: 'shardB',
  date: ISODate('2026-04-08T01:51:46.744Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t:
Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775613053, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:30:13.080Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611801, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611801, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
```

```
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:30:13.126Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:30:13.632Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27028',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 2036,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1775611813, i: 1 }),
      electionDate: ISODate('2026-04-08T01:30:13.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27028',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1304,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastHeartbeat: ISODate('2026-04-08T01:51:45.185Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:51:46.136Z'),
      pingMs: Long('0'),
```

```
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27028',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27028',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1304,
      optime: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613103, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:51:43.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:51:43.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:51:43.256Z'),
      lastHeartbeat: ISODate('2026-04-08T01:51:45.185Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:51:46.124Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27028',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775613103, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775613103, i: 1 })
}
shardB [direct: primary] test>
```

Shard C

```
[ec2-user@ip-172-31-69-137 ~]$ mongosh --port 27038
Current Mongosh Log ID: 69d5b4d09cc3a7090844ba88
Connecting to:
mongodb://127.0.0.1:27038/?directConnection=true&serverSelectionTimeoutMS=2
000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-08T01:17:52.225+00:00: Access control is not enabled for the
database. Read and write access to data and configuration is unrestricted
------

shardC [direct: primary] test> rs.status()
{
  set: 'shardC',
  date: ISODate('2026-04-08T01:52:20.726Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t:
Long('1') },
    lastCommittedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t:
Long('1') },
```

```
    appliedOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
    durableOpTime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
    lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
    lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1775613122, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-04-08T01:33:22.638Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1775611991, i: 1
}), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1775611991, i: 1 }), t:
Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-04-08T01:33:22.681Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-04-08T01:33:23.189Z')
  },
  members: [
    {
      _id: 0,
      name: '172.31.69.137:27038',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 2069,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      electionTime: Timestamp({ t: 1775612002, i: 1 }),
      electionDate: ISODate('2026-04-08T01:33:22.000Z'),
      configVersion: 1,
      configTerm: 1,
      self: true,
```

```
      lastHeartbeatMessage: ''
    },
    {
      _id: 1,
      name: '172.31.77.233:27038',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1148,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastHeartbeat: ISODate('2026-04-08T01:52:20.697Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:52:19.691Z'),
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27038',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    },
    {
      _id: 2,
      name: '172.31.76.188:27038',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 1148,
      optime: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1775613132, i: 1 }), t: Long('1')
},
      optimeDate: ISODate('2026-04-08T01:52:12.000Z'),
      optimeDurableDate: ISODate('2026-04-08T01:52:12.000Z'),
      lastAppliedWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastDurableWallTime: ISODate('2026-04-08T01:52:12.823Z'),
      lastHeartbeat: ISODate('2026-04-08T01:52:20.683Z'),
      lastHeartbeatRecv: ISODate('2026-04-08T01:52:19.669Z'),
      pingMs: Long('1'),
```

```
      lastHeartbeatMessage: '',
      syncSourceHost: '172.31.69.137:27038',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 1,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1775613132, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1775613132, i: 1 })
}
shardC [direct: primary] test>
```

Task 14

node1 (172.31.69.137):
 configsvr PRIMARY @27019
 shardA PRIMARY @27018
 shardB PRIMARY @27028
 shardC PRIMARY @27038
 mongos @27017

node2 (172.31.77.233):
 configsvr SECONDARY @27019
 shardA SECONDARY @27018
 shardB SECONDARY @27028
 shardC SECONDARY @27038

node3 (172.31.76.188):
 configsvr SECONDARY @27019
 shardA SECONDARY @27018
 shardB SECONDARY @27028

shardC SECONDARY @27038